UNIT V: Association Rule Mining

Q1. What is Frequent Pattern Mining? Explain Frequent Itemsets, Closed Itemsets, and Association Rules with Example.

Frequent pattern mining is a key data mining technique used to discover **frequent patterns, associations, correlations, or causal structures** among sets of items in transactional or relational databases.

Its primary goal is to find itemsets that appear **frequently together** in a dataset.

Term	Explanation
Frequent Itemsets	Sets of items that appear together in transactions with frequency (support) above a threshold.
Closed Itemsets	Frequent itemsets where none of its supersets has the same support count. They represent maximal information without redundancy.
Association Rules	Implication expressions of the form A⇒BA \Rightarrow B, meaning if itemset AA occurs, then itemset BB is likely to occur as well.

Example: Consider a transaction dataset:

Transaction ID	Items Bought
1	Bread, Milk
2	Bread, Diaper, Beer
3	Milk, Diaper, Beer
4	Bread, Milk, Diaper
5	Bread, Milk, Diaper, Beer

- **Frequent Itemset:** {Bread, Milk} appears in transactions 1, 4, 5 (support = 3/5 = 60%)
- **Closed Itemset:** {Bread, Milk, Diaper} appears in transactions 4 and 5 (support = 2/5 = 40%) and none of its supersets have same support.
- **Association Rule:** {Diaper} ⇒ {Beer} with support = 3/5 = 60% and confidence = support({Diaper, Beer}) / support({Diaper}) = 3/4 = 75%.

Summary:

- **Frequent Itemsets** are the foundation of association rule mining.
- **Closed Itemsets** provide a compact representation avoiding redundancy.
- **Association Rules** help discover meaningful relationships between items in large datasets.

Q2. Write Down the Difference Between Apriori and Eclat

Criteria	Apriori Algorithm	Eclat Algorithm
Approach	Uses a **breadth-first search** to generate candidate itemsets level by level	Uses a **depth-first search** to explore itemsets recursively
Data Representation	Works on horizontal data format (transaction list)	Works on vertical data format (tid-lists or transaction IDs)
Candidate Generation	Generates candidate itemsets explicitly	Uses intersections of tid-lists to find frequent itemsets
Memory Usage	Generally higher due to candidate generation	More memory efficient due to tid-lists and recursion
Performance	Slower due to multiple scans of database and candidate generation	Faster for dense datasets due to efficient intersection operations
Complexity	High computational cost in candidate generation	More efficient in handling large datasets

	and pruning	
Pruning	Uses Apriori property: all subsets of a frequent itemset must be frequent	Prunes candidates using tid-list intersections
Implementation	Easier to implement	Slightly complex due to recursive processing

---

Q3. Define FP-Growth. What Are the Advantages of FP-Growth over Apriori Algorithm?

FP-Growth is a **frequent pattern mining** algorithm that **efficiently discovers frequent itemsets** without candidate generation. It uses a **compact data structure called FP-Tree (Frequent Pattern Tree)** to represent the database, and mines the frequent patterns directly from this tree.

How FP-Growth Works:

1. **Build FP-Tree:** Scan the database once to find frequent items (above minimum support). Sort frequent items in descending order of support. Create an FP-Tree by inserting transactions, sharing common prefixes.
2. **Mine FP-Tree:** Extract frequent itemsets by recursively exploring conditional FP-Trees (subtrees conditioned on a particular item).

Advantages of FP-Growth over Apriori:

Feature	FP-Growth	Apriori
Candidate Generation	No candidate generation	Generates large numbers of candidate itemsets
Database Scans	Requires only two scans	Multiple scans for each level of itemsets
Data Structure	Uses compact FP-Tree to compress data	Uses simple list-based representation
Performance	Faster and more scalable for large datasets	Slower due to expensive candidate generation
Memory Efficiency	More memory efficient with compressed data	Uses more memory for candidates and support counts
Complexity	Handles dense and long transactions better	Struggles with dense datasets and long frequent patterns

FP-Growth is a highly efficient frequent pattern mining algorithm that overcomes Apriori's limitations by avoiding candidate generation and compressing data into a compact tree structure, resulting in improved speed and scalability.

Q4. Explain Apriori Algorithm with Example

Apriori is a classical algorithm used for **frequent itemset mining** and **association rule learning** over transactional databases. It works on the principle that **all subsets of a frequent itemset must also be frequent** (Apriori property).

Working Steps:

1. **Scan the database** to find all frequent 1-itemsets (items with support ≥ minimum support).
2. **Generate candidate (k)-itemsets** from frequent (k-1)-itemsets using self-join.
3. **Prune** candidate itemsets whose subsets are not frequent.
4. **Scan the database** to count the support of candidates.
5. **Select frequent k-itemsets** with support ≥ minimum support.
6. **Repeat steps 2-5** until no more frequent itemsets are found.
7. **Generate association rules** from frequent itemsets with confidence ≥ minimum confidence.

Example: Consider a dataset with transactions:

Transaction ID	Items Bought
1	Bread, Milk
2	Bread, Diaper, Beer

3	Milk, Diaper, Beer
4	Bread, Milk, Diaper
5	Bread, Milk, Diaper, Beer

- Minimum Support = 60% (3 transactions)

Step 1: Find frequent 1-itemsets:

Item	Support Count
Bread	4
Milk	4
Diaper	4
Beer	3

Step 2: Generate candidate 2-itemsets: {Bread, Milk}, {Bread, Diaper}, {Bread, Beer}, {Milk, Diaper}, {Milk, Beer}, {Diaper, Beer}

Count support and select those ≥ 3.

- {Bread, Milk} = 3
- {Bread, Diaper} = 3
- {Milk, Diaper} = 3
- {Diaper, Beer} = 3
- Others less than 3 → pruned

Step 3: Generate candidate 3-itemsets from frequent 2-itemsets: {Bread, Milk, Diaper}, {Milk, Diaper, Beer}, {Bread, Diaper, Beer}

Count support:

- {Bread, Milk, Diaper} = 2 (below 3, discard)
- {Milk, Diaper, Beer} = 2 (discard)
- {Bread, Diaper, Beer} = 2 (discard)

No frequent 3-itemsets.

Q5. What is Association Rule? What Are the Applications of Association Rule Mining?

Association rule mining discovers interesting relationships, patterns, or correlations between sets of items in large datasets. An **association rule** is typically expressed as: $A \Rightarrow B$

where A and B are disjoint itemsets. The rule implies that the presence of itemset A in a transaction suggests the likely presence of itemset B.

Key Measures of Association Rules:

- **Support:** The proportion of transactions containing both A and B.
  $\text{Support}(A \Rightarrow B) = \text{Count}(A \cup B) / \text{Total Transactions}$
- **Confidence:** The likelihood that BB appears in transactions containing AA.
  $\text{Confidence}(A \Rightarrow B) = \text{Count}(A \cup B) / \text{Count}(A)$
- **Lift:** Measures the strength of the rule over random chance.
  $\text{Lift}(A \Rightarrow B) = \text{Confidence}(A \Rightarrow B) / \text{Support}(B)$

Applications of Association Rule Mining:

Application Area	Description
Market Basket Analysis	Understanding customer purchase behavior to identify product combinations frequently bought together.
Cross-Selling and Upselling	Suggesting related products to customers based on their purchase history.
Inventory Management	Optimizing stock based on frequent item combinations.
Web Usage Mining	Discovering patterns in web page visits to improve navigation and recommendations.

Fraud Detection	Identifying suspicious patterns in transactions.
Healthcare	Discovering relationships between symptoms and diseases for diagnosis and treatment planning.
Telecommunications	Detecting frequent call or message patterns for targeted marketing or network optimization.

Q6. Discuss Support, Confidence, and Lift in Details

1. Support: Support of an association rule A⇒BA \Rightarrow B measures how frequently the itemset A∪BA \cup B appears in the dataset. It reflects the overall significance of the rule.

Formula: Support(A ⇒ B) = Number of transactions containing (A∪B) / Total number of transactions
Interpretation: Higher support means the rule is applicable to a larger part of the dataset.

2. Confidence: Confidence indicates the likelihood of occurrence of BB in transactions that contain AA. It reflects the strength of the implication.

Formula: Confidence(A ⇒ B) = Number of transactions containing A∪B / Number of transactions containing A
Interpretation: A higher confidence implies that when AA occurs, BB is very likely to occur.

3. Lift: Lift measures how much more likely BB occurs with AA than if AA and BB were independent.

Formula: Lift(A ⇒ B) = Confidence(A ⇒ B) / Support(B)
Interpretation:

- Lift > 1: Positive correlation between AA and BB (rule is useful)
- Lift = 1: AA and BB are independent
- Lift < 1: Negative correlation (occurrence of AA reduces likelihood of BB)

Q7. Compare and Contrast FP-Growth and Apriori Algorithm

Criteria	FP-Growth Algorithm	Apriori Algorithm
Approach	Uses a compact data structure called FP-Tree to store data	Generates candidate itemsets level-wise
Candidate Generation	Does **not** generate candidate itemsets explicitly	Generates large sets of candidate itemsets
Database Scans	Requires **two scans** of the database	Requires multiple scans, one for each level of itemsets
Data Representation	Uses **vertical tree** structure to compress transactions	Uses horizontal transaction list
Efficiency	More efficient and faster, especially for large datasets	Less efficient, slower with large or dense datasets
Memory Usage	Uses more memory to store FP-Tree but less for candidate sets	Uses more memory for storing and counting candidate sets
Handling of Long Patterns	Efficient with long frequent patterns	Struggles with long frequent patterns
Implementation Complexity	More complex due to tree construction and recursive mining	Relatively simpler to implement

- **FP-Growth** is more scalable and efficient for large datasets as it avoids candidate generation by using a compact tree structure.
- **Apriori** is easier to understand but can be computationally expensive due to repeated candidate generation and database scanning.
